

Projet GLPI multi-site

Rapport complet et detaille

Traitement et administration des donnees

Etablissement	CY Tech
Projet	Refonte simplifiee d'une base GLPI sous Oracle XE
Perimetre	Campus de Cergy et Pau
Auteurs	Gournay Lucas, Chapin Florian, De Morais Theo

17 mai 2026

Table des matières

1	Introduction	2
1.1	Contexte general	2
1.2	Objectifs du projet	2
1.3	Plan suivi	2
2	Analyse du probleme initial	2
2.1	Complexite du modele GLPI	2
2.2	Limites d'un modele trop proche de GLPI	3
2.3	Strategie de simplification	3
3	Reverse engineering et modelisation	3
3.1	Domaines fonctionnels conserves	3
3.2	Comparaison avant et apres simplification	4
3.3	MCD du projet	4
3.4	Justification des cardinalites	5
4	Modele logique de donnees	5
4.1	Principes du MLD	5
4.2	Organisation	5
4.3	Referentiels	6
4.4	Utilisateurs, profils et groupes	6
4.5	Inventaire	7
4.6	Support	7
4.7	Reseau	8
4.8	Tables systeme	9
5	Architecture repartie BDDR	9
5.1	Principe general	9
5.2	Fragmentation horizontale	10
5.3	Schemas, vues et synonymes	10
5.4	Vues globales	11
5.5	Transfert inter-sites	11
6	Integrite, securite et droits	11
6.1	Contraintes relationnelles	11
6.2	Roles Oracle	11
6.3	Audit et archivage	12
7	Optimisation physique	12
7.1	Tablespaces	12
7.2	Clusters	12
7.3	Index B-tree et composites	13
7.4	Index fonctionnels et bitmap	13
8	Vues metier	13
8.1	Role des vues	13
8.2	Exemple : inventaire complet	13

9	Logique metier PL/SQL	13
9.1	Procedures	13
9.2	Fonctions	14
9.3	Triggers	14
9.4	Interet pedagogique du PL/SQL	14
10	Jeu de donnees et demonstration	14
10.1	Generation des donnees	15
10.2	Scenario de demonstration	15
11	Analyse des performances	15
11.1	Protocole	15
11.2	Requetes representatives	15
11.3	Interpretation attendue	16
12	Limites et ameliorations possibles	16
12.1	Limites du projet	16
12.2	Ameliorations	16
13	Conclusion	16
A	Annexe A - Dictionnaire synthetique des tables	16
B	Annexe B - Ordre d'execution des scripts	17

1 Introduction

1.1 Contexte general

GLPI est une solution de gestion de parc informatique et de support. Son schema relationnel reel est tres riche, car il couvre un grand nombre de modules : inventaire, tickets, contrats, reseau, utilisateurs, droits, plugins et historique. Cette richesse devient cependant difficile a exploiter dans le cadre d'un mini-projet de base de donnees repartie : le modele initial comporte beaucoup de tables, peu de relations explicites faciles a presenter et plusieurs structures tres proches pour des objets materiels differents.

Le projet propose donc une refonte simplifiee d'une base inspiree de GLPI pour deux campus, Cergy et Pau. L'objectif n'est pas de reproduire toute la base GLPI, mais de construire un noyau coherent, contraint, explicable et demonstrable sous Oracle XE. La conception conserve les notions importantes : organisation multi-sites, utilisateurs, profils, groupes, inventaire, tickets, reseau, audit, archivage, optimisation physique et simulation BDDR.

1.2 Objectifs du projet

Le projet poursuit cinq objectifs principaux, repris du plan de presentation :

1. realiser un reverse engineering du modele GLPI pour identifier les entites utiles au contexte pedagogique ;
2. produire un modele simplifie, lisible et justifie par un MCD et un MLD ;
3. proposer une architecture de base de donnees repartie entre Cergy et Pau ;
4. ajouter une logique metier PL/SQL garantissant l'integrite et les traitements principaux ;
5. optimiser les acces avec tablespaces, clusters, index et mesures de performance.

1.3 Plan suivi

Le rapport suit le plan de la presentation :

Partie de presentation	Traitement dans le rapport
Contexte et objectifs	Perimetre, limites de GLPI et objectifs Oracle.
Reverse engineering et modelisation	Choix de simplification, MCD, MLD et contraintes.
Architecture repartie	Fragmentation horizontale, schemas Cergy/Pau, DB links et vues globales.
Optimisation physique	Tablespaces, clusters, index B-tree, composites, fonctionnels et bitmap.
Logique metier PL/SQL	Procedures, fonctions, triggers, audit et securite.
Analyse des performances	Protocole de benchmark et interpretation attendue.

2 Analyse du probleme initial

2.1 Complexite du modele GLPI

Le schema GLPI original contient plusieurs centaines de tables. Beaucoup de modules ne sont pas necessaires pour le perimetre du projet : contrats, licences, notifications, calendriers avances, plugins, historique detaille, relations reseau tres fines, etc. Conserver tout ce schema aurait rendu la base difficile a expliquer et a distribuer proprement.

Le premier enjeu a donc ete de separer le coeur fonctionnel du bruit technique. Le coeur retenu correspond aux besoins suivants :

- connaitre les sites et localisations physiques ;
- gerer les utilisateurs et leurs droits principaux ;
- inventorier les equipements informatiques ;
- suivre les incidents ou demandes lies a un materiel ;
- associer les materiels a des ports reseau et a des adresses IP ;
- tracer les modifications importantes.

2.2 Limites d'un modele trop proche de GLPI

Un modele trop proche de GLPI aurait pose plusieurs difficultes :

- multiplication des tables de materiel, par exemple ordinateurs, ecrans, imprimantes, telephones, peripheriques et equipements reseau ;
- nombreuses associations polymorphes, difficiles a représenter avec des clés étrangères simples ;
- vues et requetes plus lourdes, car l'inventaire complet doit etre reconstruit par unions ou jointures multiples ;
- distribution moins lisible, car chaque famille d'objets aurait son propre fragment ;
- demonstration plus fragile, puisque le jury doit comprendre rapidement le modele et ses choix.

2.3 Strategie de simplification

La simplification principale consiste a centraliser tous les equipements dans une table unique `assets`. La colonne `asset_type` indique la nature du materiel :

Type	Sens fonctionnel
COMPUTER	Ordinateur fixe ou portable.
MONITOR	Ecran.
PERIPHERAL	Peripherique general.
PRINTER	Imprimante.
PHONE	Telephone.
NETWORK_EQUIPMENT	Switch, borne Wi-Fi, routeur ou equipement reseau.

Ce choix remplace plusieurs tables quasiment identiques par une structure commune. Il reduit les jointures, simplifie les vues metier et rend l'ajout d'un nouveau type de materiel possible sans modifier le schema.

3 Reverse engineering et modelisation

3.1 Domaines fonctionnels conserves

Le modele final contient 19 tables, dont 17 tables fonctionnelles et 2 tables systeme. Elles sont reparties en six domaines :

Domaine	Tables
Organisation	<code>sites</code> , <code>locations</code> .
Referentiels	<code>manufacturers</code> , <code>states</code> , <code>ticket_categories</code> .

Utilisateurs et droits	users, profiles, profiles_users, groups, groups_users.
Inventaire	assets.
Support	tickets, ticket_users, ticket_followups.
Reseau	network_ports, ip_networks, ip_addresses.
Systeme	audit_log, archives_materiel.

3.2 Comparaison avant et apres simplification

Probleme du modele source	Choix du modele final
Tables distinctes pour chaque type de materiel.	Une seule table assets , typee par asset_type .
Relations polymorphes dans les tickets et les ports reseau.	Une seule cle etrangere asset_id .
Nombreux referentiels utilisateurs secondaires.	Conservation des profils, groupes et affectations essentielles.
Modele reseau tres fin.	Conservation des ports, sous-reseaux, VLAN et adresses IP.
Historique disperse.	Tables systeme dediees pour l'audit et l'archivage.

3.3 MCD du projet

Le MCD represente les entites retenues et leurs associations principales. Il met volontairement en avant la table centrale **assets**, reliee aux sites, aux utilisateurs, aux localisations, aux fabricants, aux etats, aux tickets et aux ports reseau.

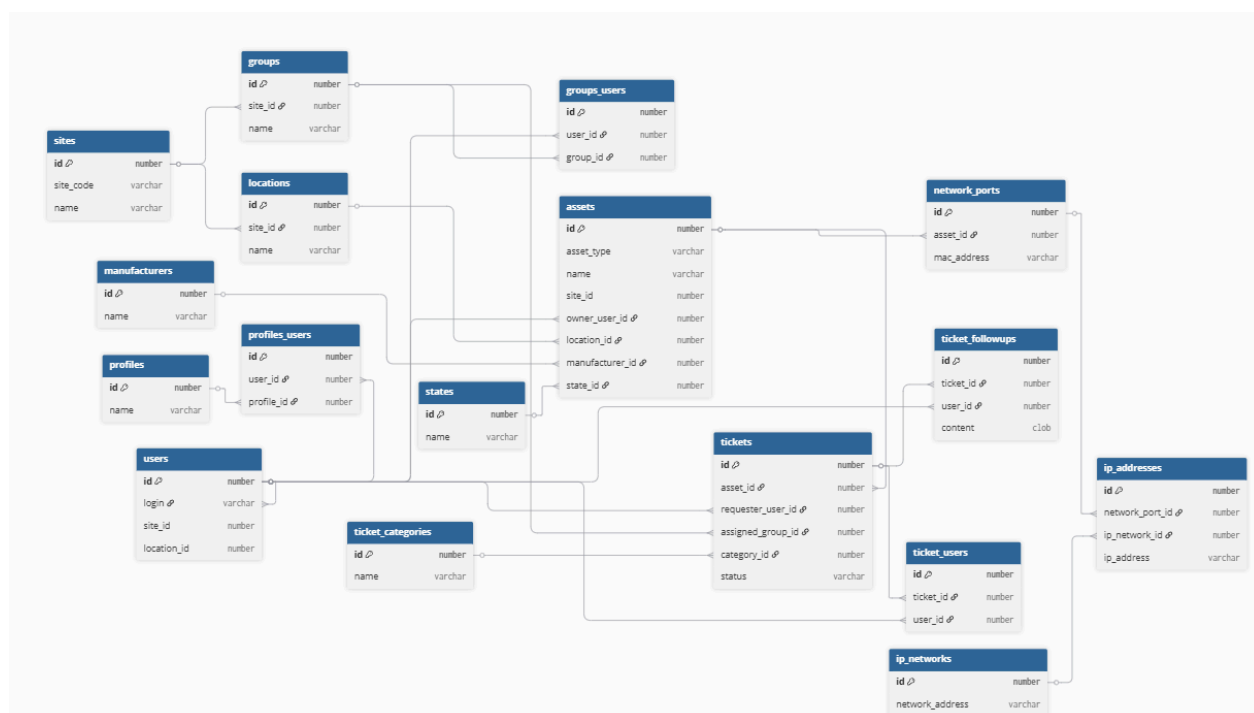


FIGURE 1 – MCD simplifié de la base GLPI multi-site.

La lecture du MCD se fait autour de quatre axes :

- un site possède des localisations, des utilisateurs, des groupes, des équipements, des tickets et des éléments réseau ;
- un utilisateur peut posséder un matériel, être technicien responsable, demander un ticket ou intervenir dans un suivi ;
- un matériel peut avoir plusieurs tickets et plusieurs ports réseau ;
- un ticket possède des affectations de techniciens et des suivis.

3.4 Justification des cardinalités

Relation	Cardinalité	Justification
<code>sites -> locations</code>	1,N	Un campus ou service peut contenir plusieurs bâtiments ou salles.
<code>sites -> users</code>	1,N	Un utilisateur appartient à un site principal.
<code>sites -> assets</code>	1,N	Chaque équipement est rattaché à un site propriétaire.
<code>assets -> tickets</code>	1,N	Un matériel peut générer plusieurs demandes support.
<code>assets -> network_ports</code>	1,N	Un équipement peut exposer plusieurs interfaces réseau.
<code>network_ports -> ip_addresses</code>	1,N	Un port peut recevoir une ou plusieurs adresses selon le contexte.
<code>tickets -> ticket_followups</code>	1,N	Un ticket contient un historique de commentaires.
<code>users -> profiles_users</code>	1,N	Un utilisateur peut avoir plusieurs profils selon le site.

4 Modèle logique de données

4.1 Principes du MLD

Le MLD traduit le MCD en tables relationnelles Oracle. Les choix structurants sont les suivants :

- chaque table possède une clé primaire numérique `id` ;
- les relations sont exprimées par des clés étrangères lisibles ;
- les tables de liaison portent les associations N,N ;
- les doublons fonctionnels sont évités par des contraintes **UNIQUE** ;
- les suppressions dépendantes sont encadrées avec **ON DELETE CASCADE** ou **ON DELETE SET NULL** selon le sens métier.

4.2 Organisation

```
sites(
  id PK,
  name,
  parent_site_id FK -> sites.id,
  full_name,
```

```

    site_code,
    created_at,
    updated_at
)

locations(
    id PK,
    site_id FK -> sites.id,
    name,
    parent_location_id FK -> locations.id,
    full_name,
    building,
    room,
    created_at,
    updated_at
)

```

`sites` porte la hierarchie logique et le code de distribution `site_code`. `locations` represente les lieux physiques, comme un batiment ou une salle.

4.3 Referentiels

```

manufacturers(id PK, name UNIQUE)
states(id PK, name UNIQUE)
ticket_categories(id PK, name UNIQUE, description)

```

Ces tables sont stables et peu volumineuses. Elles sont donc de bonnes candidates a la replication logique dans les deux schemas de simulation BDDR.

4.4 Utilisateurs, profils et groupes

```

users(
    id PK,
    login UNIQUE,
    last_name,
    first_name,
    email,
    phone,
    site_id FK -> sites.id,
    location_id FK -> locations.id,
    supervisor_user_id FK -> users.id,
    is_active,
    created_at,
    updated_at
)

profiles(id PK, name UNIQUE, interface, is_default)

profiles_users(
    id PK,
    user_id FK -> users.id,
    profile_id FK -> profiles.id,
    site_id FK -> sites.id,
    is_recursive,
    UNIQUE(user_id, profile_id, site_id)
)

groups(

```



```

    id PK,
    site_id FK -> sites.id,
    name,
    parent_group_id FK -> groups.id,
    full_name,
    created_at,
    updated_at
)

groups_users(
    id PK,
    user_id FK -> users.id,
    group_id FK -> groups.id,
    is_manager,
    UNIQUE(user_id, group_id)
)

```

Le couple `profiles` et `profiles_users` reprend l'idée GLPI d'affectation de profils selon un contexte. Les groupes servent principalement à représenter les équipes support locales.

4.5 Inventaire

```

assets(
    id PK,
    site_id FK -> sites.id,
    asset_type,
    name,
    serial_number,
    owner_user_id FK -> users.id,
    technician_user_id FK -> users.id,
    location_id FK -> locations.id,
    manufacturer_id FK -> manufacturers.id,
    state_id FK -> states.id,
    created_at,
    updated_at,
    UNIQUE(site_id, serial_number)
)

```

`assets` est la table centrale du projet. La contrainte `UNIQUE(site_id, serial_number)` évite deux numéros de série identiques sur un même site, tout en laissant la possibilité théorique qu'un même serial soit réutilisé dans un autre fragment si le cas métier le permet.

4.6 Support

```

tickets(
    id PK,
    site_id FK -> sites.id,
    asset_id FK -> assets.id ON DELETE CASCADE,
    title,
    description,
    status,
    priority,
    requester_user_id FK -> users.id,
    assigned_group_id FK -> groups.id,
    category_id FK -> ticket_categories.id,
    resolution,
    created_at,

```

```

    updated_at,
    assigned_at,
    resolved_at,
    closed_at
)

ticket_users(
    id PK,
    ticket_id FK -> tickets.id ON DELETE CASCADE,
    user_id FK -> users.id,
    assigned_by_user_id FK -> users.id,
    assigned_at,
    UNIQUE(ticket_id, user_id)
)

ticket_followups(
    id PK,
    ticket_id FK -> tickets.id ON DELETE CASCADE,
    user_id FK -> users.id,
    content,
    created_at
)

```

Le support est volontairement centre sur un materiel. Un ticket est associe a un `asset_id`, ce qui simplifie l'analyse des incidents et la creation des vues metier.

4.7 Réseau

```

network_ports(
    id PK,
    site_id FK -> sites.id,
    asset_id FK -> assets.id ON DELETE CASCADE,
    port_name,
    mac_address,
    port_type,
    created_at,
    updated_at
)

ip_networks(
    id PK,
    site_id FK -> sites.id,
    network_name,
    network_address,
    subnet_mask,
    gateway_address,
    vlan_name,
    vlan_tag,
    created_at,
    updated_at,
    UNIQUE(site_id, network_address, subnet_mask)
)

ip_addresses(
    id PK,
    site_id FK -> sites.id,
    network_port_id FK -> network_ports.id ON DELETE SET NULL,
    ip_network_id FK -> ip_networks.id,
    ip_address,

```

```
created_at,  
updated_at  
)
```

Le reseau est limite aux elements utiles pour une demonstration : ports, sous- reseaux, VLAN et adresses IP. Cette granularite suffit pour montrer des jointures et des statistiques sans introduire un modele reseau trop lourd.

4.8 Tables systeme

```
audit_log(  
  id PK,  
  table_name,  
  row_id,  
  action,  
  old_data,  
  new_data,  
  changed_by,  
  changed_at  
)  
  
archives_materiel(  
  id PK,  
  original_table,  
  original_id,  
  archived_data,  
  archived_by,  
  archived_at  
)
```

`audit_log` conserve les modifications significatives. `archives_materiel` garde une trace JSON d'un materiel avant suppression definitive.

5 Architecture repartie BDDR

5.1 Principe general

La BDDR est simulee localement sous Oracle XE avec deux schemas : `GLPI_CERGY` et `GLPI_PAU`. Chaque schema expose le fragment logique de son site. Cette simulation garde les principes d'une base repartie tout en restant executable sur un seul poste.

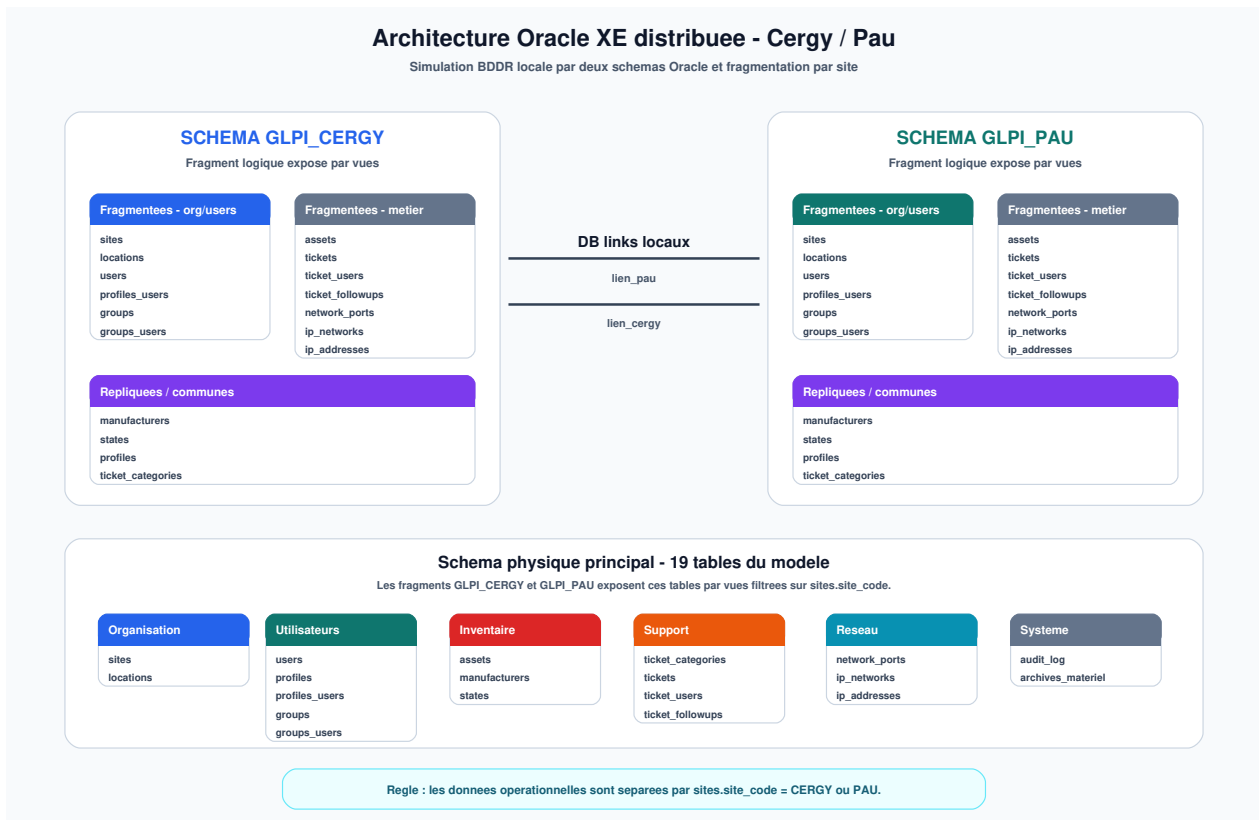


FIGURE 2 – Architecture Oracle XE distribuee entre Cergy et Pau.

5.2 Fragmentation horizontale

La fragmentation repose sur `sites.site_code`. Les donnees operationnelles sont decoupees selon le site :

Famille de donnees	Strategie
Sites et localisations	Chaque schema expose les lignes de son site.
Utilisateurs et groupes	Fragmentation par site de rattachement.
Assets	Fragmentation par <code>assets.site_id</code> .
Tickets et suivis	Fragmentation selon le site du ticket et du materiel.
Reseau	Ports, sous-reseaux et IP filtres par site.
Referentiels	Exposition commune dans les deux schemas.

Ce choix est naturel : Cergy heberge les equipements, utilisateurs, tickets et adresses de Cergy ; Pau heberge ceux de Pau. Les referentiels sont repliques car ils changent rarement et doivent rester disponibles des deux cotes.

5.3 Schemas, vues et synonymes

Le script `07_bddr.sql` cree les utilisateurs `glpi_cergy` et `glpi_pau`, puis expose des vues locales dans chaque schema. Les vues permettent de masquer le schema source et d'offrir une interface logique comme si chaque site possedait ses propres tables.

Les database links `lien_pau` et `lien_cergy` permettent ensuite d'interroger le site distant. Des synonymes, par exemple `assets_pau` ou `assets_cergy`, masquent la syntaxe distante `@lien_pau` ou `@lien_cergy`.

5.4 Vues globales

Les vues globales fournissent une vision consolidee :

Vue globale	Role
V_ASSETS_GLOBAL	Inventaire Cergy + Pau avec indication du fragment.
V_USERS_GLOBAL	Utilisateurs des deux sites.
V_STATS_GLOBAL	Statistiques agregees par site.
V_TICKETS_GLOBAL	Tickets consolides.

Cette approche montre le principe classique d'une BDDR : les applications locales travaillent sur leur fragment, tandis que les besoins de reporting passent par des vues globales.

5.5 Transfert inter-sites

La procedure `SP_TRANSFERT_MATERIEL` represente le traitement cle de la BDDR. Son role est de deplacer un materiel d'un site vers l'autre. Dans une situation locale, le changement peut se limiter a la mise a jour de `assets.site_id`. Dans un cas inter-sites, le transfert doit tenir compte du fragment distant :

1. identifier le site courant du materiel ;
2. identifier le site cible ;
3. recopier ou mettre a jour le materiel dans le fragment distant ;
4. deplacer les dependances utiles, notamment ports reseau, adresses IP, tickets, affectations et suivis ;
5. supprimer l'ancien fragment local lorsque le transfert est termine.

Cette logique illustre le probleme central des BDDR : une operation metier simple en apparence peut impliquer plusieurs objets repartis et doit preserver l'integrite referentielle.

6 Integrite, securite et droits

6.1 Contraintes relationnelles

L'integrite est d'abord assuree par le schema relationnel :

- les cles primaires identifient chaque ligne ;
- les cles etrangeres imposent la coherence entre domaines ;
- les contraintes `UNIQUE` evitent les doublons fonctionnels ;
- les contraintes `CHECK` encadrent les valeurs enumerees comme `asset_type`, `status` ou `priority`.

La contrainte `UNIQUE(site_id, serial_number)` sur `assets` remplace avantageusement un trigger de verification de serial, car elle evite les problemes de table mutante et laisse Oracle garantir l'unicite directement.

6.2 Roles Oracle

Le projet definit quatre roles principaux :

Role	Responsabilite
ROLE_ADMIN	Administration complete du schema et des objets.

ROLE_TECHNICIEN	Consultation et actions sur le support, l'inventaire et le reseau.
ROLE_CONSULTANT	Lecture des vues metier.
ROLE_MANAGER_SITE	Suivi des donnees et indicateurs du site.

Ces roles separent les usages : administration, intervention technique, consultation et pilotage local.

6.3 Audit et archivage

L'audit est realise par TRG_AUDIT_ASSETS, qui journalise les operations sur les materiels. L'archivage est realise par TRG_ARCHIVE_ASSET_DELETE, qui conserve l'etat final d'un materiel avant suppression.

Ce mecanisme repond a deux besoins :

- garder une trace des modifications importantes ;
- pouvoir expliquer ce qui est arrive a un materiel supprime ou transfere.

7 Optimisation physique

7.1 Tablespaces

Le projet separe les donnees dans plusieurs tablespaces :

Tablespace	Utilisation
TS_MATERIEL	Inventaire et donnees associees aux assets.
TS_UTILISATEURS	Utilisateurs, profils et groupes.
TS_RESEAU	Ports, sous-reseaux et adresses IP.
TS_SUPPORT	Tickets et suivis support.
TS_INDEX	Index et structures d'optimisation.

Cette separation rend l'organisation physique plus claire et permet de presenter une logique d'administration Oracle : stockage par domaine, surveillance des volumes et isolation des index.

7.2 Clusters

Les clusters regroupent physiquement des lignes souvent lues ensemble. Le projet utilise trois clusters :

Cluster	Cle	Tables concernees
cluster_user_rights	user_id	users, profiles_users, groups_users.
cluster_site_structure	site_id	sites, locations, groups, ip_networks.
cluster_network_port_ips	network_port_id	network_ports, ip_addresses.

Le cluster `cluster_user_rights` est particulierement utile pour expliquer l'objectif : lorsqu'on consulte un utilisateur, ses profils et groupes sont des donnees frequemment accedees ensemble. Les regrouper physiquement peut reduire le nombre de lectures disque.

7.3 Index B-tree et composites

Les index B-tree couvrent les cles étrangères et les recherches fréquentes. Les index composites ciblent les requêtes métier :

- `idx_assets_site_type` pour l'inventaire par site et type ;
- `idx_assets_site_state` pour l'inventaire par site et état ;
- `idx_assets_site_owner` pour retrouver le matériel d'un utilisateur ;
- `idx_ticket_site_status_priority` pour suivre les tickets ouverts ou prioritaires par site ;
- `idx_ticket_asset_status` pour analyser les tickets d'un matériel ;
- `idx_network_site_address` pour les sous-réseaux d'un site.

7.4 Index fonctionnels et bitmap

Les index fonctionnels facilitent les recherches insensibles à la casse : `idx_assets_name_upper`, `idx_assets_serial_upper`, `idx_users_login_upper` et `idx_users_email_upper`.

Les index bitmap sont utilisés sur des colonnes à faible cardinalité, comme `asset_type`, `is_active`, `priority` et `port_type`. Ils sont intéressants pour les requêtes analytiques, notamment sur de gros volumes, mais doivent être présentes avec prudence dans un contexte très transactionnel.

8 Vues métier

8.1 Role des vues

Les vues servent à fournir une abstraction métier au-dessus du modèle relationnel. Elles évitent de demander aux utilisateurs ou au jury de retenir toutes les jointures.

Vue	Utilité
V_INVENTAIRE_COMPLET	Inventaire enrichi avec site, localisation, propriétaire, technicien, fabricant et état.
V_MATERIEL_PAR_SITE	Comptage des matériels par site et par type.
V_UTILISATEURS_PROFILS	Utilisateurs avec profils et site d'application.
V_TOPOLOGIE_RESEAU	Ports, adresses IP, sous-réseaux et VLAN.
V_STATISTIQUES_SITE	Indicateurs synthétiques par site.
V_MATERIEL_RECENT	Matériels modifiés récemment.
V_TICKETS_SUPPORT	Tickets enrichis avec demandeur, catégorie, groupe et matériel.

8.2 Exemple : inventaire complet

`V_INVENTAIRE_COMPLET` est la vue la plus importante pour la démonstration. Elle montre directement l'intérêt de la table `assets` : une seule vue suffit pour lister l'ensemble du parc, quel que soit le type de matériel.

9 Logique métier PL/SQL

9.1 Procédures

Procédure	Rôle
-----------	------

SP_TRANSFERT_MATERIEL	Transfert local ou inter-sites d'un materiel.
SP_AFFECTER_PROFIL	Affectation ou mise a jour d'un profil utilisateur.
SP_CREER_TICKET_MATERIEL	Creation d'un ticket sur un asset existant du site.
SP_ASSIGNER_TECH_TICKET	Affectation d'un technicien a un ticket.
SP_INVENTAIRE_SITE	Affichage synthetique de l'inventaire d'un site.
SP_NETTOYER_ARCHIVES	Suppression des archives trop anciennes.
SP_GENERER_JEU_DE_TEST	Generation d'un jeu de donnees representatif.
SP_REPLIQUER_REFERENTIELS	Procedure de demonstration liee a la BDDR.

9.2 Fonctions

Fonction	Role
FN_COMPTER_MATERIEL_SITE	Compte les assets d'un site.
FN_CALCULER_TAUX_UTILISATION	Calcule le taux d'utilisation reseau d'un asset.
FN_OBTENIR_SITE_ENTITE	Retrouve le code site associe a une entite.
FN_VERIFIER_QUOTA_MATERIEL	Verifie si un quota de materiel est depasse.

9.3 Triggers

Trigger	Table	Role
TRG_AUTO_DATE_MOD_ASSETS	assets	Mise a jour de <code>updated_at</code> .
TRG_AUTO_DATE_MOD_USERS	users	Mise a jour de <code>updated_at</code> .
TRG_AUTO_DATE_MOD_NETPORTS	network_ports	Mise a jour de <code>updated_at</code> .
TRG_AUTO_DATE_MOD_TICKETS	tickets	Gestion des dates du workflow ticket.
TRG_CHECK_TICKET_USER_TECH	ticket_users	Controle qu'un technicien affecte possede le bon profil.
TRG_AUDIT_ASSETS	assets	Journalisation des modifications d'inventaire.
TRG_ARCHIVE_ASSET_DELETE	assets	Archivage JSON avant suppression.
TRG_CASCADE_SITE_CODE	sites	Propagation d'un changement de code site.

9.4 Interet pedagogique du PL/SQL

Le PL/SQL montre que la base ne se limite pas a stocker des donnees. Elle porte aussi des regles :

- automatiser les dates de modification ;
- empecher certaines affectations incoherentes ;
- produire des indicateurs ;
- encapsuler les operations sensibles comme la creation d'un ticket ou le transfert d'un materiel.

10 Jeu de donnees et demonstration

10.1 Generation des donnees

Le script `09_test_data.sql` cree la procedure `SP_GENERERER_JEU_DE_TEST`. Elle genere un volume de donnees suffisant pour tester les vues et les performances :

- sites Cergy et Pau ;
- localisations ;
- utilisateurs ;
- profils, groupes et affectations ;
- assets de plusieurs types ;
- tickets, techniciens et suivis ;
- ports reseau, sous-reseaux et adresses IP.

Le jeu de test permet de montrer les requetes sans saisir manuellement des dizaines de lignes pendant la soutenance.

10.2 Scenario de demonstration

Une demonstration efficace peut suivre cet ordre :

1. presenter le MCD et expliquer la centralite de **assets** ;
2. montrer le MLD et les contraintes principales ;
3. executer les vues metier, notamment `V_INVENTAIRE_COMPLET` et `V_TICKETS_SUPPORT` ;
4. creer un ticket avec `SP_CREER_TICKET_MATERIEL` ;
5. afficher l'audit ou les dates modifiees par trigger ;
6. montrer les schemas `GLPI_CERGY` et `GLPI_PAU` ;
7. interroger une vue globale BDDR ;
8. expliquer le benchmark avec index visibles puis invisibles.

11 Analyse des performances

11.1 Protocole

Le protocole presente dans les slides repose sur l'idee suivante :

1. injecter un volume important de donnees avec `SP_GENERERER_JEU_DE_TEST` ;
2. vider ou neutraliser autant que possible les effets du cache Oracle pour obtenir des mesures comparables ;
3. executer les requetes representatives ;
4. comparer les resultats avec index visibles et index invisibles ;
5. stocker les mesures dans **benchmark_results**.

11.2 Requetes representatives

Les benchmarks visent les usages principaux :

- recherche d'un asset par nom ;
- recherche par numero de serie ;
- inventaire d'un site par type ;
- tickets ouverts par site, statut et priorite ;
- topologie reseau d'un materiel ;
- droits d'un utilisateur via profils et groupes.

11.3 Interpretation attendue

Les index doivent ameliorer surtout les recherches selectives et les filtres frequents. Les index composites sont utiles quand les colonnes sont utilisees ensemble, par exemple **site_id**, **status** et **priority** pour les tickets. Les clusters deviennent interessants lorsque plusieurs tables sont lues ensemble de maniere repetelee, comme les droits utilisateurs.

Le resultat attendu n'est pas seulement un temps plus faible. Il faut aussi montrer que chaque index correspond a un usage metier. Un index inutile coute de l'espace et ralentit les insertions ; un bon index accelere une requete importante et justifie son cout de maintenance.

12 Limites et ameliorations possibles

12.1 Limites du projet

Le projet reste une simplification. Certaines fonctionnalites completes de GLPI ne sont pas modelisees :

- contrats, fournisseurs, licences logicielles et budgets ;
- historique complet de tous les objets ;
- droits fins comparables au moteur GLPI reel ;
- synchronisation BDDR reelle entre deux serveurs physiques ;
- resolution automatique des conflits inter-sites.

12.2 Ameliorations

Plusieurs evolutions sont possibles :

- separer la demonstration locale et une version multi-instances Oracle ;
- ajouter des materialized views pour certains rapports globaux ;
- enrichir l'audit avec le contexte applicatif ;
- ajouter une table de mouvements d'inventaire pour tracer les transferts ;
- comparer les plans d'execution avant et apres chaque index ;
- automatiser la generation d'un rapport de benchmark.

13 Conclusion

Le projet aboutit a une base Oracle coherente et defendable. La refonte simplifiee conserve les idees essentielles de GLPI tout en supprimant la complexite non necessaire au cadre pedagogique. La table **assets** rend l'inventaire lisible, les vues fournissent une abstraction metier, la BDDR illustre la fragmentation Cergy/Pau, et le PL/SQL ajoute les regles de gestion attendues.

La proposition est donc pertinente pour une soutenance : elle montre a la fois la modelisation, l'administration Oracle, la distribution des donnees, la securite, l'integrite, les traitements stockes et l'optimisation des performances. Le modele n'est pas une copie exhaustive de GLPI ; c'est une version volontairement recentree, plus facile a expliquer et a tester.

A Annexe A - Dictionnaire synthetique des tables

Table	Description
sites	Sites, campus et structures logiques.

<code>locations</code>	Emplacements physiques rattaches aux sites.
<code>manufacturers</code>	Fabricants des equipements.
<code>states</code>	Etats fonctionnels des equipements.
<code>users</code>	Utilisateurs, demandeurs, techniciens et responsables.
<code>profiles</code>	Profils applicatifs.
<code>profiles_users</code>	Affectation des profils aux utilisateurs selon le site.
<code>groups</code>	Groupes de support ou d'organisation.
<code>groups_users</code>	Membres des groupes.
<code>assets</code>	Inventaire centralise des materiels.
<code>ticket_categories</code>	Categories de tickets.
<code>tickets</code>	Demandes et incidents.
<code>ticket_users</code>	Techniciens affectes aux tickets.
<code>ticket_followups</code>	Historique des suivis de tickets.
<code>network_ports</code>	Ports reseau des assets.
<code>ip_networks</code>	Sous-reseaux IP et VLAN.
<code>ip_addresses</code>	Adresses IP rattachees aux ports.
<code>audit_log</code>	Journal d'audit.
<code>archives_materiel</code>	Archives JSON des materiels supprimees.

B Annexe B - Ordre d'execution des scripts

```

01_tablespace.sql
02_schema_tables.sql
04_clusters_indexes.sql
03_users_roles.sql
05_views.sql
06_plsql/triggers.sql
06_plsql/procedures.sql
06_plsql/functions.sql
06_plsql/cursors.sql
09_test_data.sql
08_query_plans.sql
10_benchmark.sql
07_bddr.sql

```